Universidad Tecmilenio	**Gestión de proyectos de inteligencia artificial**
	Docente: Tania Judith Mendoza Torres
	AL02568360 Ricardo Alejandro Lastra Dueñas
	Fase II. Ejecución ❖ Documentación de validación y pruebas

1. Documentación de validación y pruebas

El objetivo de esta documentación es verificar que WODPredict-AI funciona como una solución tecnológica funcional y no solo como modelo de machine learning experimental. Que el contenedor construye la imagen correctamente y arranca sin errores. Que el servicio inicie la API, queda operativa y responda de forma correcta. Que sea consistente, donde entradas iguales produzcan resultados iguales bajo las mismas condiciones. Que sea verificable con entradas normales y extremas, que rechace datos inválidos y que el tiempo de respuesta es viable para el uso diario del coach. Las métricas del modelo (recall, precisión, ROC-AUC) ya se validaron previamente y se comprobó que WODPredict-AI sabe predecir; aquí se valida el comportamiento del modelo dentro del sistema para que el coach pueda utilizarlo. La prueba de infraestructura verifica que Docker pueda construir y ejecutar sin errores la solución WODPredict-AI: construir la imagen, iniciar el contenedor, comprobar la disponibilidad de la API, verificar que no existan errores en la inicialización. En la prueba funcional de la API comprobamos que reciba los datos correctamente que ingresó el coach, que procese los datos, ejecute el modelo y devuelva la respuesta esperada. La prueba de valores extremos verificamos que soporte valores cercanos dentro de los límites establecidos. Por ejemplo, RPE debe estar entre un rango mínimo de 1 y máximo de 10. El objetivo es que el sistema no se bloque bajo estos escenarios con valores extremos. Con la prueba de validación de datos comprobamos que la API rechace entradas incorrectas como valores fuera de rango, tipo de dato incorrecto, valores inválidos (por ejemplo, un valor negativo).

2. Entorno de pruebas

Elemento	Detalle
Imagen	wodpredict-api:1.0 (python:3.12-slim, ≈ 690 MB en disco)
Construcción	docker build -t wodpredict-api:1.0 . (sin errores)
Ejecución	docker run -d --name wodpredict -p 8000:8000 wodpredict-api:1.0, estado healthy
Servicio accesible	http://localhost:8000/ (health check), /app (formulario), /docs (Swagger)
Modelo	Regresión logística v1.0.0, umbral 0.17
Herramientas	pytest 9.2, requests 2.32, notebook fase2_ejecucion.ipynb (VS Code)
Equipo	Windows 11, AMD Ryzen 5 7535HS (3.30 GHz, 12 hilos), 32 GB de RAM; Docker Desktop 4.97 (Docker Engine 29.9.2); Python 3.12.16 en un ambiente conda
Datos de prueba	3 perfiles en tests/ejemplos/ (riesgo bajo, moderado y alto) y variaciones de ellos

3. Estrategia de pruebas

Tipo	Ejemplo	Qué demuestra	Casos
Funcional (F)	Entrada válida → respuesta 200	El flujo principal funciona	7
Caso extremo (E)	Valores vacíos, mínimos o máximos	Comportamiento ante situaciones difíciles	5
Validación de entrada (V)	Tipo incorrecto, fuera de rango o faltante	La API rechaza entradas inválidas	9
Consistencia y portabilidad (C)	Misma entrada, mismo modelo; antes y después de Docker	Inferencia reproducible y entorno portable	2
Latencia (L)	Tiempo por petición	Viabilidad operacional	1

Todos los casos se ejecutan de dos formas: en el notebook (puntos 6 a 8), que arma la tabla esperado contra real, y como pruebas automatizadas en tests/test_api.py, que cualquier persona puede repetir con pytest tests/ -v contra el contenedor local o contra la URL de la nube.

4. Pruebas funcionales

ID	Entrada	Esperado	Real	Resultado
F01	GET /	200 y "API funcionando"	200 y "API funcionando"	Pasa
F02	GET /modelo	Umbral 0.17 y recall ≥ 0.70	Umbral 0.17, recall 0.732	Pasa
F03a	POST /predict, perfil riesgo bajo	200, sin alerta, riesgo Bajo	200, p = 0.030, Bajo	Pasa
F03b	POST /predict, perfil riesgo moderado	200, alerta, riesgo Moderado	200, p = 0.303, Moderado	Pasa
F03c	POST /predict, perfil riesgo alto	200, alerta, riesgo Alto	200, p = 0.750, Alto	Pasa
F04	GET /app	200, formulario que llama a /predict	200, contiene fetch("/predict")	Pasa
F05	GET /docs y /openapi.json	200 y /predict documentado	200, /predict en el contrato	Pasa

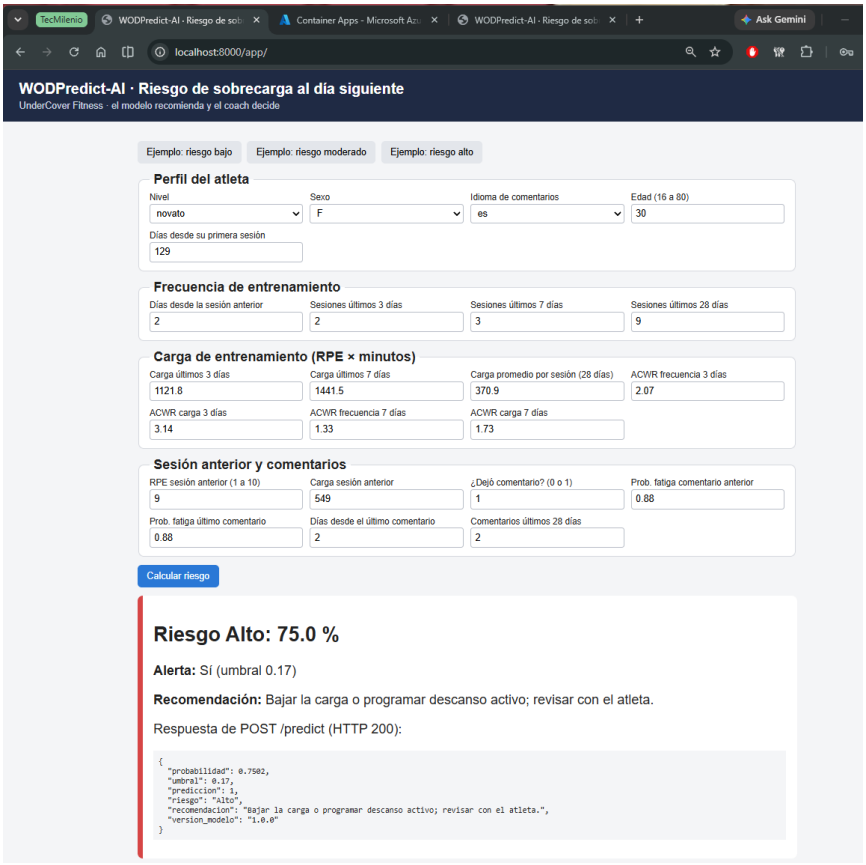


Figura 1. Prueba de integración frontend-API: el formulario /app envía el perfil de riesgo alto a POST /predict y muestra la respuesta real (HTTP 200).

5. Casos extremos

ID	Entrada	Esperado	Real	Resultado
E01	Sin edad, RPE, ACWR ni comentarios (10 campos en null)	200 y probabilidad válida (imputación)	200, p = 0.229, Moderado	Pasa
E02	Mínimos: regresa tras 365 días, todo en 0, RPE 1	200 y riesgo Bajo	200, p < 0.001, Bajo	Pasa

E03	Máximos: carga 20,000, ACWR 4, RPE 10, fatiga 1	200 y riesgo Alto	200, p = 1.000, Alto	Pasa
E04	Solo sube la carga de 7 días (300 a 3,000)	La probabilidad no baja	0.054 → 0.140 → 0.423 → 0.768	Pasa
E05	Campo extra no esperado ("nombre_atleta")	200, se ignora	200, p = 0.303, Moderado	Pasa

WODPredict-AI no funciona nada mas con datos normales, sino que también responde correctamente con situaciones atípicas o extremas que se le pueden presentar a un coach en el día a día. E01 demuestra que el sistema puede trabajar con atletas nuevos que les falta información, por lo que utiliza la mediana calculada durante el entrenamiento del modelo para sustituir esos datos faltantes y generar una predicción válida. Las pruebas E02 y E03 introducen respectivamente valores al extremo inferior y extremo superior dentro del rango permitido. Estas pruebas demuestran que el modelo es capaz de procesar valores extremos y generar resultados sin errores. La prueba E04 verifica si el modelo tiene sentido desde la perspectiva del entrenamiento deportivo, que el sistema responde de manera coherente cuando subimos la carga e identifica cuando debe emitir una alerta al coach. Con estas pruebas verificamos la confiabilidad en condiciones reales de WODPredict-AI como sistema, no solamente por sus buenas métricas (ROC-AU y recall) de machine learning.

6. Validación de entrada

ID	Entrada	Esperado	Real	Resultado
V01	prev_rpe = 11	422 en prev_rpe	422, "less than or equal to 10"	Pasa
V02	level = "pro"	422 en level	422, valores permitidos	Pasa
V03	load_last_7d = "mucha"	422 en load_last_7d	422, "valid number"	Pasa
V04	prob_fatiga_last_note = 1.5	422 en prob_fatiga_last_note	422, "≤ 1"	Pasa
V05	sessions_last_7d = -1	422 en sessions_last_7d	422, "≥ 0"	Pasa
V06	age = 120	422 en age	422, "≤ 80"	Pasa
V07	Falta load_last_7d	422 tipo missing	422, "Field required"	Pasa
V08	JSON mal formado	422	422, json_invalid	Pasa
V09	GET /predict (método incorrecto)	405	405, Method Not Allowed	Pasa

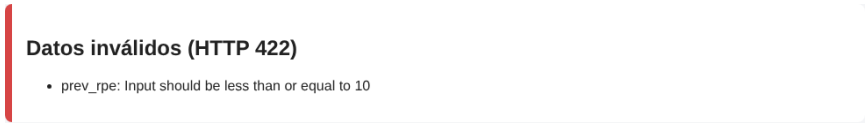


Figura 2. El formulario muestra el error 422 cuando el RPE es 11: la API señala el campo y no ejecuta el modelo.

7. Consistencia y portabilidad

Perfil	Modelo directo (joblib)	API local (uvicorn)	API en Docker	¿Iguales?
Riesgo bajo	0.0300	0.0300	0.0300	Sí
Riesgo moderado	0.3025	0.3025	0.3025	Sí
Riesgo alto	0.7502	0.7502	0.7502	Sí

Estos análisis en WODPredict-AI comprueban que sus predicciones sean reproducibles y que sigue funcionando no importe donde se ejecute. C01 (consistencia): la misma entrada enviada 20 veces devolvió siempre la misma probabilidad (30.25%). Esta prueba demuestra que el modelo es reproducible. C02 (portabilidad): las probabilidades son idénticas con el modelo cargado y ejecutado directamente desde Python (escenario 1), con la API ejecutada desde la terminal en el entorno local (escenario 2) y con la API dentro del contenedor (escenario 3), lo que demuestra que Docker reproduce el mismo entorno. Esta prueba

demuestra que el modelo es portable. Por lo tanto, el modelo no depende las diferencias entre los ambientes de ejecución.

8. Latencia

Se enviaron 200 peticiones seguidas a POST /predict en el contenedor local. Se simuló el uso de la API como si un coach o una aplicación solicitara repetidamente predicciones. La mediana fue de 17.3 ms y el percentil 95 de 22.4 ms, muy por debajo de la meta. Los valores corresponden a la ejecución del punto 8 del notebook en un equipo local. Esta prueba demuestra que WODPredict-AI genera predicciones correctas, pero además las entrega rápidamente.

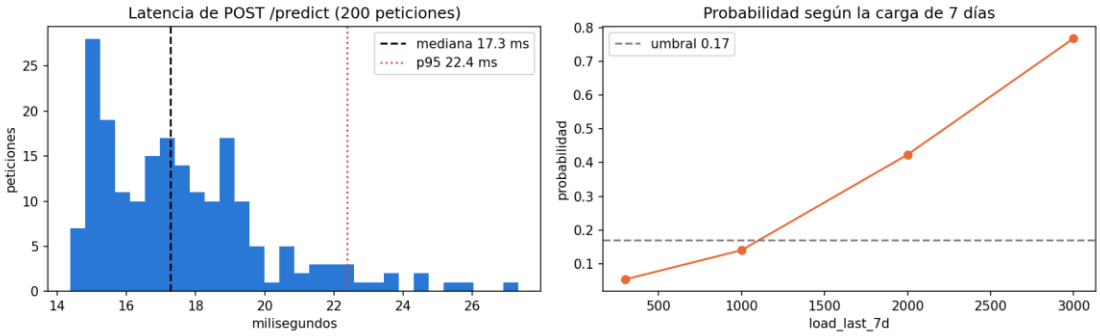


Figura 3. Distribución de la latencia de POST /predict (izquierda) y probabilidad según la carga de 7 días (derecha).

9. Pruebas automatizadas (pytest)

Con la herramienta de Python, pytest, se ejecutaron pruebas automatizadas para verificar los diferentes componentes del sistema. La batería completa se ejecutó desde el punto 9 del notebook contra el contenedor Docker local: 23 pruebas aprobadas en 3.33 s, sin fallos. Esta herramienta demuestra que todos los componentes evaluados responden correctamente, es una evidencia de calidad y confiabilidad. Además respalda que la integración del modelo, API y Docker funcionan correctamente en el entorno local.

```
tests/test_api.py::test_F01_health_check PASSED                          [  4%]
tests/test_api.py::test_F02_ficha_del_modelo PASSED                      [  8%]
tests/test_api.py::test_F03_prediccion_por_perfil[bajo-0-Bajo] PASSED    [ 13%]
tests/test_api.py::test_F03_prediccion_por_perfil[moderado-1-Moderado] PASSED [ 17%]
tests/test_api.py::test_F03_prediccion_por_perfil[alto-1-Alto] PASSED    [ 21%]
tests/test_api.py::test_F04_frontend_disponible PASSED                   [ 26%]
tests/test_api.py::test_F05_documentacion_swagger PASSED                 [ 30%]
tests/test_api.py::test_E01_datos_opcionales_vacios PASSED               [ 34%]
tests/test_api.py::test_E02_limites_minimos PASSED                       [ 39%]
tests/test_api.py::test_E03_limites_maximos PASSED                       [ 43%]
tests/test_api.py::test_E04_mas_carga_mas_riesgo PASSED                  [ 47%]
tests/test_api.py::test_E05_campos_extra_se_ignoran PASSED               [ 52%]
tests/test_api.py::test_V01_valores_invalidos[prev_rpe-11] PASSED        [ 56%]
tests/test_api.py::test_V01_valores_invalidos[level-pro] PASSED          [ 60%]
tests/test_api.py::test_V01_valores_invalidos[load_last_7d-mucha] PASSED [ 65%]
tests/test_api.py::test_V01_valores_invalidos[prob_fatiga_last_note-1.5] PASSED [ 69%]
tests/test_api.py::test_V01_valores_invalidos[sessions_last_7d--1] PASSED [ 73%]
tests/test_api.py::test_V01_valores_invalidos[age-120] PASSED            [ 78%]
tests/test_api.py::test_V07_campo_obligatorio_faltante PASSED            [ 82%]
tests/test_api.py::test_V08_json_mal_formado PASSED                      [ 86%]
tests/test_api.py::test_V09_metodo_incorrecto PASSED                     [ 91%]
tests/test_api.py::test_C01_misma_entrada_misma_salida PASSED            [ 95%]
tests/test_api.py::test_L01_latencia PASSED                              [100%]
============================== 23 passed in 3.33s ==============================
Salida completa guardada en evidencias/resultado_pytest.txt
```

Figura 4. Salida de pytest en el equipo del autor (punto 9 del notebook): 23 passed.

10. Validación postdespliegue en Azure

Esta validación de WODPredict-AI es para ver que no solamente funciona en una computadora o entorno local, sino que también puede desplegarse en una plataforma real de nube (Microsoft Azure), mantener sus predicciones y responder correctamente desde una URL pública. La misma imagen wodpredict-api:1.0 se publicó en Azure Container Apps y se ejecutó la batería completa contra la URL pública con https://wodpredict-api-mx.calmcliff-e9faa155.Méxicocentral.azurecontainerapps.io La imagen contiene los componentes necesarios para ejecutar WODPredict-AI: el modelo de machine learning entrenado, las dependencias de Python, la API que recibe las solicitudes y devuelve predicciones y la configuración del entorno de ejecución. Se probaron dos regiones: East US 2 (primer despliegue, después eliminado) y México Central (despliegue final) que es donde encontramos la principal diferencia entre los entornos, en la latencia.

Prueba	Local (Docker)	Azure East US 2	Azure Mexico Central
Funcionales, extremos, validación y consistencia (22 pruebas)	22 de 22	22 de 22	22 de 22
Probabilidades de los 3 perfiles (simula_consumo.py)	0.030 / 0.3025 / 0.7502	Idénticas	Idénticas
L01 latencia, mediana de 100 peticiones	17.3 ms (pasa)	838 ms (no pasa)	1,125 ms (no pasa)
Total pytest	23 de 23	22 de 23	22 de 23

Se ejecutaron 22 pruebas (funcionales, extremos, validación y consistencia) para comprobar que la API funciona correctamente. Las implementaciones en local y en las dos regiones, obtuvieron el mismo resultado, lo que confirma la portabilidad de la imagen. Esto demuestra que el despliegue en Azure no alteró el comportamiento funcional de WODPredict-AI. La prueba que falla es L01, porque exige una latencia inferior a 100 ms y esto no significa que la API esté fallando, se refiere a que no cumple el objetivo de rendimiento. Azure devuelve respuestas en 838 ms y 1,125 ms, respectivamente, no puede superar ese criterio de 100 ms, sin embargo, es suficientemente rápida para el caso de uso planteado. Esa diferencia de rendimiento en la respuesta en Azure no compromete la utilidad operativo de WODPredict-AI.

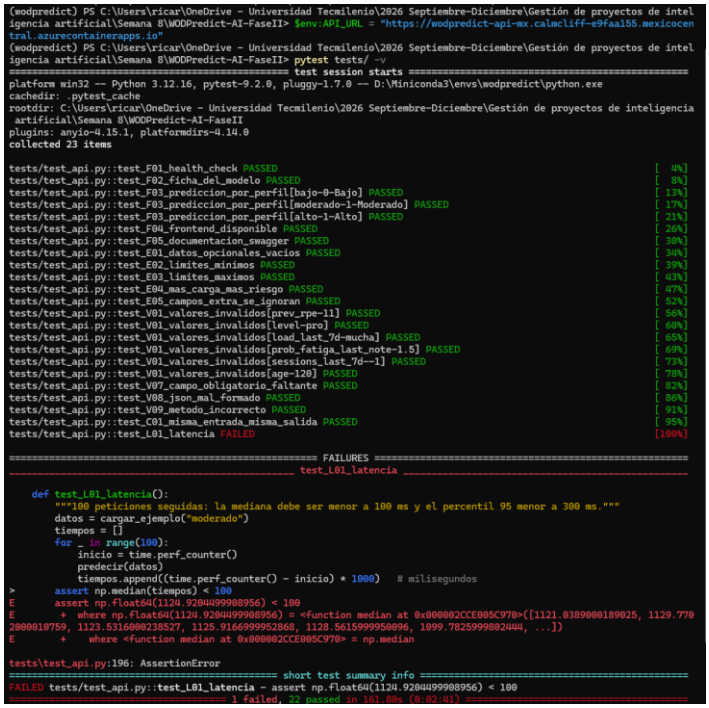


Figura 5. pytest contra la URL pública de Mexico Central: 22 passed y 1 failed (L01, mediana de 1,125 ms).

11. Resultados y conclusiones

Criterio	Resultado
Contenedor	Build sin errores, contenedor healthy en menos de 10 s
Funcionales	7 de 7
Casos extremos	5 de 5
Validación de entrada	9 de 9
Consistencia y portabilidad	2 de 2
Latencia	Mediana 17.3 ms, p95 22.4 ms
pytest	23 de 23 aprobadas
Azure Container Apps (Mexico Central)	22 de 23 (L01 fuera de la meta local; ≈ 1.1 s por consulta)

La validación post despliegue demuestra cuatro aspectos:

Portabilidad: la imagen de Docker funciona en infraestructura local y en la nube.

Integridad: las predicciones del modelo permanecen idénticas en todos los entornos probados.

Producción: pasa 22 pruebas funcionales, extremos, validación y consistencia en Azure.

Rendimiento: la latencia en Azure es aceptable para el caso de uso.

La solución cumple su función principal y es aceptable para el negocio. El coach captura los datos en el formulario y recibe en milisegundos la probabilidad, la alerta y una recomendación, con el mismo resultado que el modelo validado en actividades previas. Docker cumple su objetivo de portabilidad: el contenedor devuelve exactamente las mismas probabilidades que el entorno de desarrollo, tanto en local como desplegado en Azure. En cuanto a las limitaciones las pruebas usan perfiles sintéticos; no se probó carga concurrente de muchos usuarios ni autenticación.